

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Факультет безопасности информационных технологий

Дисциплина:

«Обеспечение информационной безопасности мобильных устройств»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №2

«Основы анализа защищенности мобильных приложений»

Выполнили:

Бардышев А. А.,

Студент группы N3349



(подпись)

Проверил:

Федоров Иван Романович,

преподаватель ФБИТ

(отметка о выполнении)

(подпись)

Санкт-Петербург

2026 г.

СОДЕРЖАНИЕ

Содержание	2
Введение	3
1 Ход работы.....	4
1.1 Insecure Logging	4
1.2 Hardcoding Issues – Part 1.....	5
1.3 Insecure Data Storage 1	6
1.4 Insecure Data Storage 2.....	8
1.5 Insecure Data Storage 3	9
1.6 Insecure Data Storage 4	11
1.7 Input Validation Issues 1	12
1.8 Input Validation Issues 2.....	13
1.9 Access Control Issues 1	13
1.10 Access Control Issues 2	14
1.11 Access Control Issues 3	16
1.12 Hardcoding Issues 2.....	17
1.13 Input Validation Issues 3	19
Вывод.....	21

ВВЕДЕНИЕ

Цель работы – изучение уязвимостей мобильных приложений на платформе Android и получение практических навыков анализа безопасности.

Для достижения поставленной цели необходимо решить следующие задачи:

- Собрать стенд для пентестига Android;
- выполнить все кейсы в Diva Android на эмуляторе Android;
- оформить все кейсы в отчет.

1 ХОД РАБОТЫ

Для начала соберем стенд для пентестинга Android с помощью Genymotion, выбрав в качестве эмулятора рекомендованную модель Nexus 5. Установим приложение Damn Insecure Vulnerable App (DIVA) — крайне уязвимое тестовое приложение, созданное для изучения типичных уязвимостей, характерных для Android-платформы (рисунок 1).

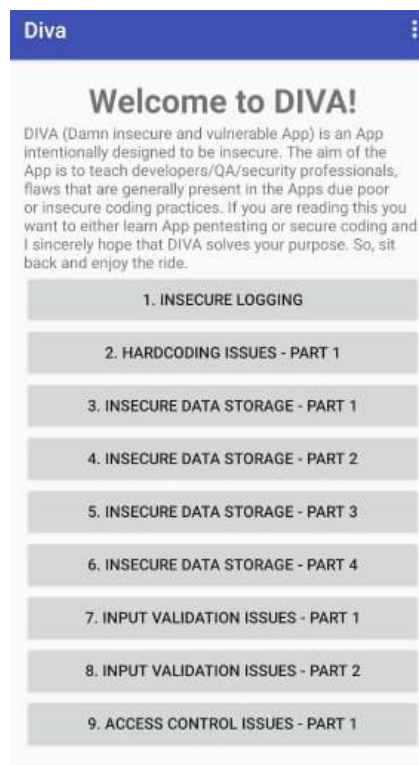


Рисунок 1 – Начальный экран

1.1 Insecure Logging

Уязвимость небезопасного логирования возникает, когда в системные журналы случайно попадают секретные данные. Программисты часто используют логи для поиска ошибок в работе программы. Однако иногда в эти записи просачивается важная информация, которая может быть украдена, если доступ к логам получают посторонние. Чтобы защититься от такой угрозы, нужно тщательно фильтровать записываемые данные и строго контролировать доступ к журналам.

Для демонстрации проблемы откроем задание и введём вымышленный номер банковской карты (рисунок 2). Затем в программе Appie выполним команду: «adb logcat | findstr "1234"» в Appie, где 1234 – номер карты, введенный в приложение.

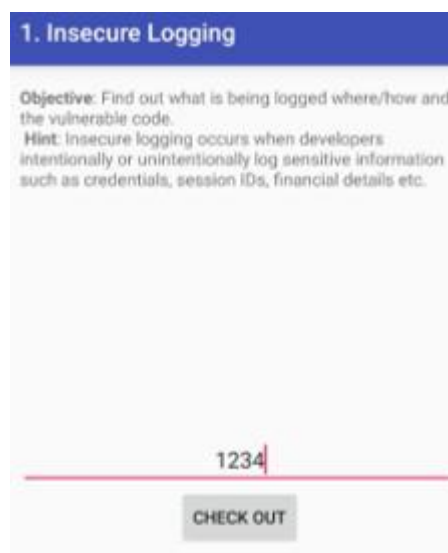


Рисунок 2 – Небезопасное логирование

```
03-02 12:05:59.598 9001 9001 E diva-log: Error while processing transaction with credit card: 1234
03-02 12:05:59.766 9001 9033 D EGL_emulation: eglMakeCurrent: 0xebb530c0: ver 2 0 (tinfo 0xebc12a20)
03-02 12:05:59.758 9001 9033 D EGL_emulation: eglMakeCurrent: 0xebb530c0: ver 2 0 (tinfo 0xebc12a20)
03-02 12:06:36.508 9001 9001 E diva-log: Error while processing transaction with credit card: 23647586
03-02 12:06:36.630 9001 9033 D EGL_emulation: eglMakeCurrent: 0xebb530c0: ver 2 0 (tinfo 0xebc12a20)
03-02 12:06:36.636 9001 9033 D EGL_emulation: eglMakeCurrent: 0xebb530c0: ver 2 0 (tinfo 0xebc12a20)
```

Рисунок 3 – Логи

Приложение DIVA записывает чувствительные данные в логи в открытом виде, что является уязвимостью (рисунки 2-3). Уязвимость Insecure Logging относится к категории Client Code Quality в OWASP Mobile Top 10.

1.2 Hardcoding Issues – Part 1

Попытаемся найти секретный ключ доступа к приложению.

Одна из задач в DIVA демонстрирует уязвимость, связанную с хардкодингом конфиденциальных данных — пароль доступа жёстко прописан в коде (рисунок 4). С помощью jadx-gui его легко обнаружить в исходном Java-файле. Такая практика небезопасна: злоумышленники могут декомпилировать приложение и получить доступ к закрытым данным.



Рисунок 4 – Экран «Hardcoding Issues – Part 1»

Изучим код приложения и найдем файл (HardcodeActivity) со строкой, где в явном виде записан код: «vendorsecretkey», с помощью которого получается получить доступ внутрь (рисунок 5).

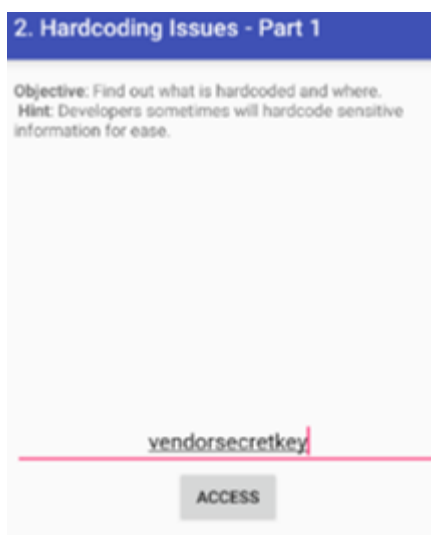


Рисунок 5 – Получение доступа внутрь

Уязвимость Hardcoding Issues относится к категории Insecure Data Storage в OWASP Mobile Top 10.

1.3 Insecure Data Storage 1

Приложение предлагает ввести данные и сообщает об их успешном сохранении (рисунки 7-8).



Рисунок 6 – Экран «Insecure Data Storage – Part 1»

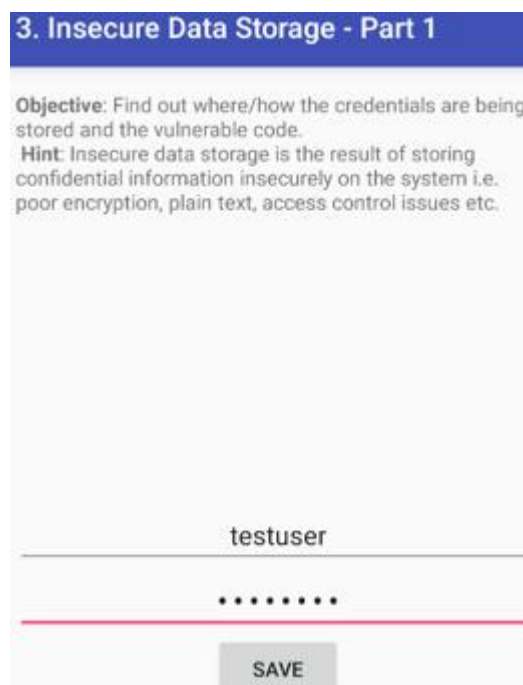


Рисунок 7 – Заполнение данными Insecure Data Storage – Part 1

При анализе кода видно, что используется метод `getDefaultSharedPreferences` для сохранения (рисунок 9).

```

19 public void saveCredentials(View view) {
20     SharedPreferences spref = PreferenceManager.getDefaultSharedPreferences(this);
21     SharedPreferences.Editor spedit = spref.edit();
22     EditText usr = (EditText) findViewById(R.id.ids1Usr);
23     EditText pwd = (EditText) findViewById(R.id.ids1Pwd);
25     spedit.putString("user", usr.getText().toString());
26     spedit.putString("password", pwd.getText().toString());
27     spedit.commit();
29     Toast.makeText(this, "3rd party credentials saved successfully!", 0).show();

```

Рисунок 8 – Анализ кода Insecure Data Storage – Part 1

Данные хранятся в директории /data/data/YOUR_PACKAGE_NAME/shared_prefs.

```
jakhar.aseem.diva_preferences.xml
generic_x86:/data/data/jakhar.aseem.diva/shared_prefs $ cat jakhar.aseem.diva_preferences.xml
at jakhar.aseem.diva_preferences.xml
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
  <string name="password">password</string>
  <string name="user">testuser</string>
</map>
generic_x86:/data/data/jakhar.aseem.diva/shared_prefs $
```

Рисунок 9 – Сохраненная информация

Переходя в неё, находим сохранённую информацию в XML-файле — это подтверждает, что данные хранятся без защиты и доступны для чтения (рисунок 10).

1.4 Insecure Data Storage 2

4. Insecure Data Storage - Part 2

Objective: Find out where/how the credentials are being stored and the vulnerable code.

Hint: Insecure data storage is the result of storing confidential information insecurely on the system i.e. poor encryption, plain text, access control issues etc.

Enter 3rd party service user name

Enter 3rd party service password

SAVE

Рисунок 10 – Экран «Insecure Data Storage – Part 2»

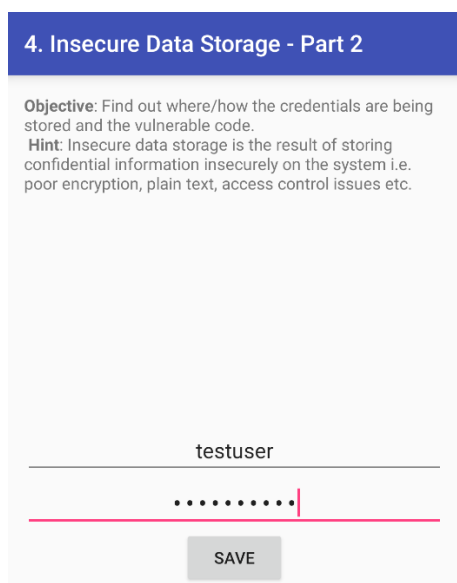


Рисунок 11 – Ввод данных для Insecure Data Storage – Part 2

Снова вводим данные, которые приложение сохраняет (рисунки 11-12).

```
public void saveCredentials(View view) {  
    EditText usr = (EditText) findViewById(R.id.ids2Usr);  
    EditText pwd = (EditText) findViewById(R.id.ids2Pwd);  
    try {  
        this.mDB.execSQL("INSERT INTO myuser VALUES ('" + usr.getText().toString() + "', '" + pwd.getText().toString() + "')");  
        this.mDB.close();  
    } catch (Exception e) {  
        Log.d("Diva", "Error occurred while inserting into database: " + e.getMessage());  
    }  
    Toast.makeText(this, "3rd party credentials saved successfully!", 0).show();  
}
```

Рисунок 12 – Анализ кода Insecure Data Storage – Part 2

В исходном коде указано, что работа с базой данных осуществляется через SQLite (рисунок 13). В папке `/data/data/YOUR_PACKAGE_NAME/` есть специальная подпапка `databases`, где хранятся четыре файла. Среди них особый интерес представляет файл `ids2` — именно туда записываются все введенные данные (рисунки 14–15).

```
generic_x86:/data/user/0/jakhar.aseem.diva/databases $ ls  
divanotes.db divanotes.db-journal ids2 ids2-journal
```

Рисунок 13 – Папка с базами данных

1.5 Insecure Data Storage 3

Временное хранение данных происходит через функцию `createTempFile`, которая создаёт специальный временный файл (рисунки 16–18). А в директории `/data/data/...` можно найти этот файл, где хранятся введенные имя и пароль (рисунок 19).



Рисунок 14 – Экран «Insecure Data Storage – Part 3»

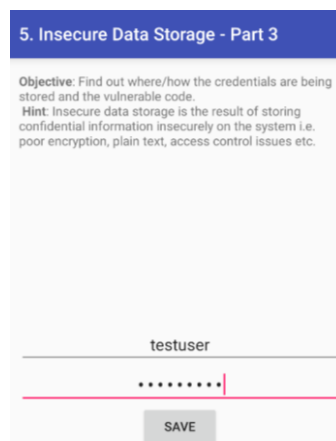


Рисунок 15 – Ввод данных для Insecure Data Storage – Part 3

```

23 public void saveCredentials(View view) {
24     EditText usr = (EditText) findViewById(R.id.ids3Usr);
25     EditText pwd = (EditText) findViewById(R.id.ids3Pwd);
27     File ddir = new File(getApplicationInfo().dataDir);
    try {
30         File uinfo = File.createTempFile("uinfo", "tmp", ddir);
31         uinfo.setReadable(true);
32         uinfo.setWritable(true);
33         FileWriter fw = new FileWriter(uinfo);
34         fw.write(usr.getText().toString() + ":" + pwd.getText().toString() + "\n");
35         fw.close();
36         Toast.makeText(this, "3rd party credentials saved successfully!", 0).show();
    } catch (Exception e) {
40         Toast.makeText(this, "File error occurred", 0).show();
41         Log.d("Diva", "File error: " + e.getMessage());
    }
}

```

Рисунок 16 – Анализ кода Insecure Data Storage – Part 3

```
generic_x86:/data/data/jakhar.aseem.diva $ cat uinfo2045144903613511269tmp
cat uinfo2045144903613511269tmp
testuser:password5
generic_x86:/data/data/jakhar.aseem.diva $ |
```

Рисунок 17 – Файлы в директории

1.6 Insecure Data Storage 4

Введем данные в приложение и попробуем их найти в памяти приложения (рисунки 20-22).

6. Insecure Data Storage - Part 4

Objective: Find out where/how the credentials are being stored and the vulnerable code.
Hint: Insecure data storage is the result of storing confidential information insecurely on the system i.e. poor encryption, plain text, access control issues etc.

testuser

.....

SAVE

Рисунок 18 – Заполнение данными Insecure Data Storage – Part 4

```
/* loaded from: classes.dex */
public class InsecureDataStorage4Activity extends AppCompatActivity {
    @Override // android.support.v7.app.AppCompatActivity, android.support.v4.app.FragmentActiv
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_insecure_data_storage4);
    }
}
```

Рисунок 19 – Анализ кода Insecure Data Storage – Part 4

```

ls -la
total 148
drwxrwx--x 12 root sdcard_rw 4096 2025-03-27 09:30 .
drwx--x--x 3 root sdcard_rw 4096 2025-03-27 09:05 ..
-rw-rw---- 1 root sdcard_rw 19 2025-03-27 09:31 .uinfo.txt
drwxrwx--x 2 root sdcard_rw 4096 2025-03-23 16:18 Alarms
drwxrwx--x 4 root sdcard_rw 4096 2025-03-23 16:18 Android
drwxrwx--x 2 root sdcard_rw 4096 2025-03-23 16:18 DCIM
drwxrwx--x 2 root sdcard_rw 4096 2025-03-23 16:18 Download
drwxrwx--x 2 root sdcard_rw 4096 2025-03-23 16:18 Movies
drwxrwx--x 2 root sdcard_rw 4096 2025-03-23 16:18 Music
drwxrwx--x 2 root sdcard_rw 4096 2025-03-23 16:18 Notifications
drwxrwx--x 2 root sdcard_rw 4096 2025-03-23 16:18 Pictures
drwxrwx--x 2 root sdcard_rw 4096 2025-03-23 16:18 Podcasts
drwxrwx--x 2 root sdcard_rw 4096 2025-03-23 16:18 Ringtones
-rw-rw---- 1 root sdcard_rw 20480 2025-03-26 17:31 divanotes.db
-rw-rw---- 1 root sdcard_rw 16384 2025-03-26 18:25 ids2
generic_x86:/sdcard $ cat .uinfo.txt
cat .uinfo.txt
testuser:password5

```

Рисунок 20 – Поиск файла с нашими данными в Insecure Data Storage 4

Сохранение во внешнее хранилище пытается использовать метод `getExternalStorageDirectory`. Без нужных разрешений это вызывает ошибку. После предоставления доступа к памяти операция проходит успешно, и на внешнем носителе появляется файл «.uinfo.txt» с сохранённой информацией.

1.7 Input Validation Issues 1

При попытке ввода текста появляется ошибка, о недостатке прав у приложения. Проверим приложение на sql injection (рисунок 23).



Рисунок 21 – Ввод инъекции

После ввода инъекции мы получаем информацию со всеми паролями и логинами. Это напрямую указывает на уязвимость к SQL-инъекциям.

1.8 Input Validation Issues 2

Попробуем ввести путь к существующему файлу (рисунок 24).

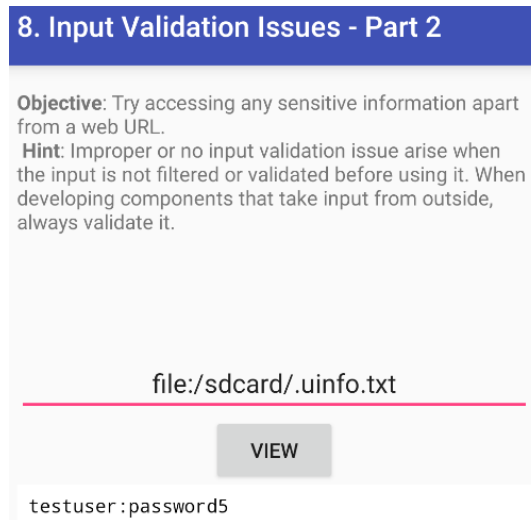


Рисунок 22 – Input Validation Issues 2

Доступ проходит успешно — появляется возможность просматривать конфиденциальную информацию, хранящуюся в различных файлах. Это уязвимость.

1.9 Access Control Issues 1

На начальном экране приложения есть кнопка «VIEW API CREDENTIALS», при нажатии на которую отображаются учётные данные для подключения к API (рисунок 25).

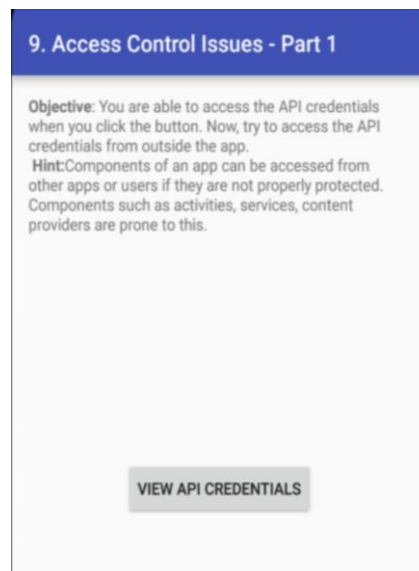


Рисунок 23 – Экран «Acces Control Issues – Part 1»

Проблема в том, что к этим данным можно получить доступ извне. Анализ logcat показывает, какая активность запускается при нажатии на кнопку (рисунок 26). Это

позволяет вручную открыть нужный экран с помощью команды: «am start jakhar.aseem.diva/.APICredsActivity».

```
Starting: Intent { act=android.intent.action.MAIN cat=[android.intent.category.LAUNCHER] cmp=jakhar.aseem.diva/.APICredsActivity }
generic x86:/ $
```

Рисунок 24 – Открытие экрана командой

После этого приложение сразу отображает все данные, без необходимости взаимодействия с интерфейсом (рисунок 27).

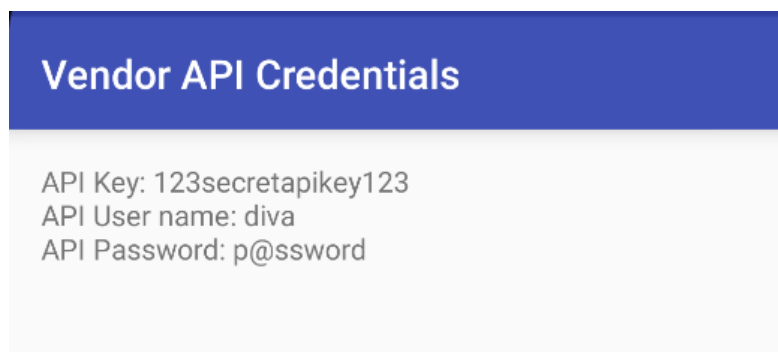


Рисунок 25 – Приложение и все данные

1.10 Access Control Issues 2

В этом случае получить доступ к критичной информации не так просто — перед отображением данных приложение требует ввод PIN-кода (рисунок 28).

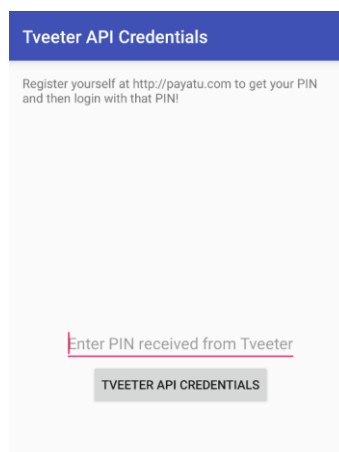


Рисунок 26 – Требование ввести PIN-код

Нажимаем кнопку «Register Now» и проверяем logcat: видим запуск активности jakhar.aseem.diva/.APICreds2Activity и запрос PIN-кода (рисунки 29-30).

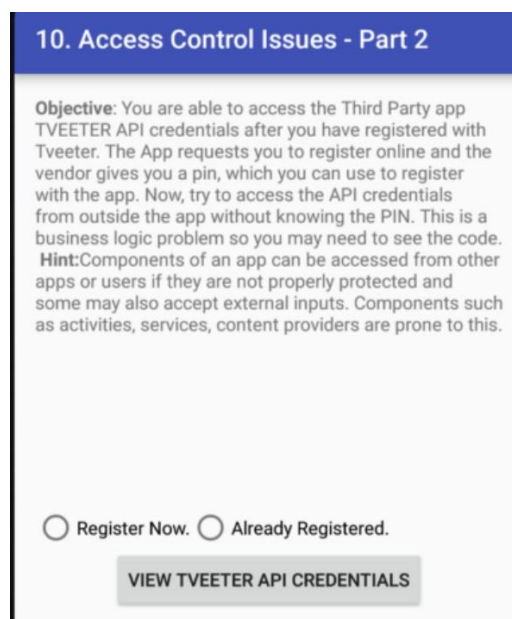


Рисунок 27 – Экран «Access Control Issues – Part 2»

```
03-27 13:39:18.183 1987 2187 I ActivityTaskManager: START u0 {act=jakhar.aseem.diva.action.VIEW_CREDS2 cmp=jakhar.aseem.diva/.APICreds2Activity (has extras)} from uid 10147
```

Рисунок 28 – Лог запуска

Анализ кода показывает, что проверка осуществляется через строку `chk_pin`, значение которой хранится в файле `res/values/strings.xml` (рисунки 31-32).

```
/* loaded from: classes.dex */
public class AccessControl2Activity extends AppCompatActivity {
    @Override // android.support.v7.app.AppCompatActivity, android.support.v4.app.Fragment
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_access_control2);
    }
}
```

Рисунок 29 – Код

```
<string name="character_counter_pattern">X11
| <string name="chk_pin">check_pin</string>
<string name="d1">1 Insecure logging</string>
```

Рисунок 30 – Значение `chk_pin`

Зная это, можно обойти проверку, выполнив команду: «adb shell am start -n jakhar.aseem.diva/.APICreds2Activity --ez check_pin false».

После этого приложение открывается и показывает данные без ввода PIN-кода (рисунки 33-34).

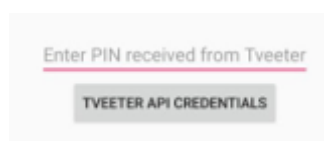


Рисунок 31 – Ввод PIN-кода для доступа внутрь

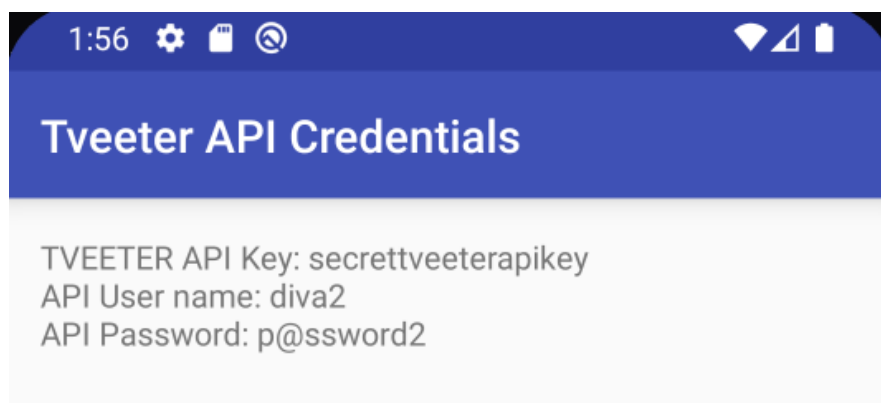


Рисунок 32 – Получение доступа

1.11 Access Control Issues 3

Задача получить данные, не вводя PIN-код. Приложение в начале предлагает создать или изменить PIN-код для доступа к личным заметкам (рисунок 35).

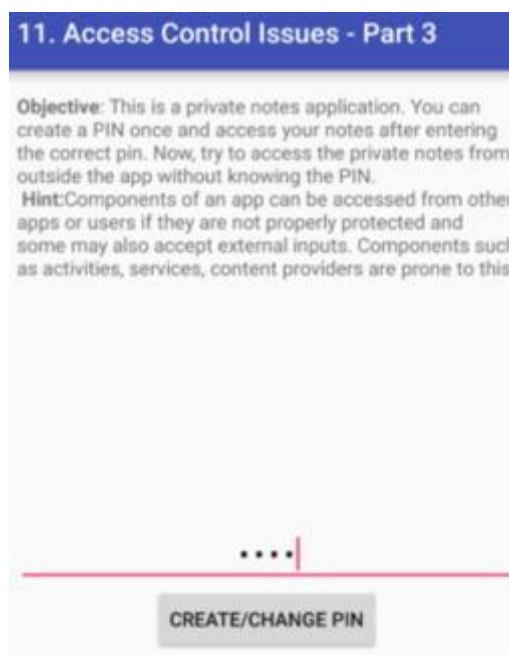


Рисунок 33 – Экран «Access Control Issues – Part 3»

В logcat отображается запуск активности jakhar.aseem.diva/.AccessControl3Activity, которая затем переходит к AccessControl3NotesActivity (рисунок 36).

```
03-27 13:49:48.991 1987 4198 I ActivityTaskManager: START u0 {cmp=jakhar.aseem.diva/.AccessControl3Activity} from uid 10147
```

Рисунок 34 – Logcat

Эта активность, в свою очередь, обращается к NotesProvider. А в файле AndroidManifest.xml видно, что NotesProvider доступен другим приложениям. В его коде указан URI для доступа к заметкам: «content://jakhar.aseem.diva.provider.notesprovider/notes».


```

/* loaded from: classes.dex */
public class AccessControl3Activity extends AppCompatActivity {
    @Override // android.support.v7.app.AppCompatActivity, android.support.v4.app.Fragment
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_access_control3);
        SharedPreferences spref = PreferenceManager.getDefaultSharedPreferences(this);
        String pin = spref.getString(getString(R.string.pkey), "");
        if (!pin.isEmpty()) {
            Button vbutton = (Button) findViewById(R.id.aci3viewbutton);
            vbutton.setVisibility(0);
        }
    }
}

```

Рисунок 35 – Код AccessControl3Activity

```

/* loaded from: classes.dex */
public class AccessControl3NotesActivity extends AppCompatActivity {
    @Override // android.support.v7.app.AppCompatActivity, android.support.v4.app.FragmentActivity, android.support.v4.app.BaseFragment
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_access_control3_notes);
    }
}

```

Рисунок 36 – Код AccessControl3NotesActivity

```

<provider
    android:name="jakhar.aseem.diva.NotesPrpvider"
    android:enabled="true"
    android:exported="true"
    android:authorities="jakhar.aseem.diva.provider.notesprovider"/>

```

Рисунок 37 – AndriodManifest.xml

```

/* loaded from: classes.dex */
public class NotesProvider extends ContentProvider {
    static final String AUTHORITY = "jakhar.aseem.diva.provider.notesprovider";
    static final String CREATE_TBL_QRY = "CREATE TABLE notes (_id INTEGER PRIMARY KEY AUTOINCREMENT, title TEXT)";
    static final String C_ID = "_id";
    static final String C_NOTE = "note";
    static final String C_TITLE = "title";
    static final String DBNAME = "divanotes.db";
    static final int DBVERSION = 1;
    static final String DROP_TBL_QRY = "DROP TABLE IF EXISTS notes";
    static final int PATH_ID = 2;
    static final int PATH_TABLE = 1;
    static final String TABLE = "notes";
    SQLiteDatabase mDB;
    static final Uri CONTENT_URI = Uri.parse("content://jakhar.aseem.diva.provider.notesprovider/notes");
    static final UriMatcher urimatcher = new UriMatcher(-1);
}

```

Рисунок 38 – Код NotesProvider

То есть анализ кода позволяет получить содержимое заметок без ввода PIN-кода, выполнив команду:

«adb shell content query --uri content://jakhar.aseem.diva.provider.notesprovider/notes» (рисунки 37-41).

1.12 Hardcoding Issues 2

Приложение снова запрашивает код, который разработчики захардкодили.

В коде активности видно обращение к классу DivaJni, который загружает нативную библиотеку divajni (рисунки 42-43).

```

// loaded from: classes.dex
public class Hardcode2Activity extends AppCompatActivity {
    private DivaJni djni;

    @Override // android.support.v7.app.AppCompatActivity, android.support.v4.app.FragmentAc
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_hardcode2);
        this.djni = new DivaJni();
    }
}

```

Рисунок 39 – Код Hardcode2Activity

```

1 package jakhar.aseem.diva;
2
3 /* loaded from: classes.dex */
4 public class DivaJni {
5     private static final String soName = "divajni";
6
7     public native int access(String str);
8
9     public native int initiateLaunchSequence(String str);
10
11     static {
12         System.loadLibrary(soName);
13     }
14 }

```

Рисунок 40 – Код DivaJni

Изучив содержимое библиотеки, находим подозрительные строки (рисунок 44).

```

file name: libdivajni.so
mime type:

00000381: __cxa_finalize
00000390: __cxa_atexit
0000039d: Java_jakhar_aseem_diva_DivaJni_access
000003c3: strncpy
000003cb: Java_jakhar_aseem_diva_DivaJni_initiateLaunchSequence
00000401: strcpy
00000408: JNI_OnLoad
00000413: libstdc++.so
00000420: libm.so
00000428: libc.so
00000430: libdl.so
00000439: _edata
00000440: __bss_start
0000044c: __bss_start__
0000045a: __bss_end__
00000466: __end__
00000473: libdivajni.so
00000658: 0lsdfgad;lh
00000668: .dotdot
00010008: GCC: (GNU) 4.9 20140827 (prerelease)
0001002e: .shstrtab
00010038: .hash
0001003e: .dynsym
00010046: .dynstr
0001004e: .rela.dyn
00010058: .rela.plt
00010062: .text
00010068: .rodata
00010070: .eh_frame_hdr
0001007e: .eh_frame
00010088: .init_array
00010094: .fini_array
000100a0: .dynamic
000100ae: .data
000100b9: .comment

```

Рисунок 41 – Содержимое библиотеки

Подставив одну из них — строку по смещению 00000658 — получаем доступ (рисунок 45).



Рисунок 42 – Получение доступ

1.13 Input Validation Issues 3

Для решения задачи необходимо вызвать переполнение, написав большое количество символов в поле для ввода (рисунок 46).

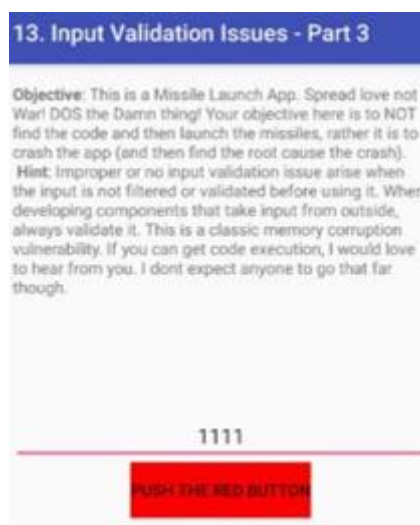


Рисунок 43 – Экран «Input Validation Issues – Part 3»

Но при вводе длинной строки (около 50 символов) происходит сбой, и выполнение останавливается (рисунок 47). Попытки найти решение по этой проблеме в DIVA не увенчались успехом — в сети нет чёткого объяснения или способа устранения ошибки.

Diva keeps stopping

-  App info
-  Close app

Рисунок 44 – Ошибка в работе приложения

ВЫВОД

В ходе выполнения лабораторной работы развернута тестовая среда Android и установлено DIVA, предназначенное для изучения типовых проблем безопасности мобильных приложений.

В процессе анализа выявлены уязвимости небезопасного логирования, позволяющие получать конфиденциальные данные из системных логов. Обнаружены случаи хардкодинга секретных ключей и PIN-кодов в исходном и нативном коде приложения. Исследованы различные способы хранения данных (SharedPreferences, базы данных SQLite, временные и внешние файлы), что позволило убедиться в отсутствии должной защиты сохранённой информации.

Также продемонстрированы уязвимости проверки входных данных, включая SQL-инъекции и возможность чтения произвольных файлов. Проведён анализ механизмов контроля доступа, показавший возможность обхода аутентификации с использованием adb-команд и прямого обращения к компонентам приложения.